lispi: a Pi kernel that's a LISP interpeter

Houjun Liu

houjun@stanford.edu

March 14, 2026

Because cursed languages work best together.

1 Some Motivation

We wanted an Emacs OS, but that required implementing libc syscalls and also just getting glib to compile, which is obviously not happening. Instead we are going for the second-best thing, which is to be able to interactively write an OS *in* Emacs.

And thus this project essentially involves enabling the process of **incremental development** of an operating system kernel. That is, beyond the bare basics and the important, atomic things, everything else should be able to be written and interpreted as they are written (e.g., instead of being compiled into the kernel before).

Of course being an Emacs purest we believe there's only one true language with which conversational programming can occur and that's LISP; thus this project essentially makes the entire Pi operating system kernel act as a little LISP interpreter which is then used to write and run drivers etc.

2 Architecture

The subjective experience of the demo involves `[laptop emacs issues command] => [thing happens on pi]`. This breaks down into a few stages:

1. **lispi.el** — `lispi-eval-at-point`, `lispi--eval-and-overlay`, and `lispi--send`; leveraging Emacs' built in sexp parsing utilities to get thing-at-point (what you are trying to send), and also subprocess utilities to talk to an unix-side UART communicator
2. **unix-side/main.rs** — a very basic loop that takes a line of user input, frames it, and tells it to the Pi
3. **shared/src/lib.rs** — the framing protocol
4. **pi-side/main.rs** — the LISP interpreter driver, takes the thing it got over UART and calls the actual evaluation; after getting a string result, it then talks it back over UART
5. **pi-side/language/execute.rs** — the actual LISP interpreter

We will expand some interesting parts and sharp bits of the sections described above, below.

3 Interpreter

The interpreter is "mostly standard" in that its designed with a usual lambda calculi evaluation semantics where we have a big match on the type of thing we see; for most atoms, we no-op and return it; for cons cells, we recurse until `car` is an atom, bail if its not a special form, and then evaluate the arguments and apply the function.

And thus there are only two main interesting things in terms of design:

3.1 LISP Image Design and Scoping

Perhaps the most interesting thing about the design of the LISP image is how we manage context. Consider:

```
(set a 12)
(set c 7)
(+ (let (a 42
           b 12)
     (set b 3)
     (set a 7)
     (set c 9)
     (+ (+ a b) c)) a)
```

We need to make sure that variables within such let-scopes (or function calls) are shadowed correctly. In order to do this, we follow a standard stack-based design for scoping environments. In particular, our environment design centers around the following four data structures in increasingly higher-level order:

```rust
/// A single mutable binding slot, shareable across multiple environments.
pub type Binding = Rc<RefCell<Rc<ast::Value>>>;

/// A flat map of symbol names to bindings.
/// Used by closures to store their captured environment snapshot.
pub type Environment = BTreeMap<String<SYMB_NAME_LEN>, Binding>;

/// A scope frame: either owned (mutable) or shared (read-only, cheap to push).
#[derive(Debug, Clone)]
enum Frame {
    /// Mutable frame for params, let-bindings, top-level set.
    Owned(Environment),
    /// Shared frame from a closure's captured env. Push is O(1) (Rc::clone).
    Shared(Rc<Environment>),
}

#[derive(Debug, Clone)]
pub struct Image {
    /// Stack of scope frames. Bottom = global, top = innermost scope.
    /// Lookups search top-to-bottom; inserts go into the top (Owned) frame.
    frames: Vec<Frame>,
}
```

At kernel start time, the bootloader builds an empty image and pushes a new owned frame onto itself `vec![Frame::Owned(BTreeMap::new())]`. The semantics of the frames are as follows:

- **owned** frames are mutable; whenever a scope is entered and bindings need to be created (e.g. for function param value bindings), we push a owned frame onto the current image stack

- **shared** frames are immutable; this is basically only used for closure context captures; this is in particular useful for stacked closures / recursions, where you maybe continuously pushing the same "captured" values again and again onto the image stack (since you for instance have a recurse closure) but early add new parameter frames

With these two primitives, scoping becomes a game of pushing and popping from the image when we enter and leave scopes. Consider the inner call region of closures:

```
// push captured env as a shared frame — O(1), just Rc::clone
image.push_env(&closure.env);
// push a fresh frame for parameter bindings
image.push_frame();

closure
    .params
    .iter()
    .zip(args.into_iter())
    .for_each(|(param, val)| {
        image.insert((**param).clone(), val);
    });

// body is always in tail position
let result = eval(Rc::clone(&closure.body), image, true)?;

image.pop_frame(); // params
image.pop_frame(); // captured env
```

We push the closed environment onto our image, and push a new owned frame where we can insert our new parameters; after evaluation, we leave scope and remove the two new frames.

3.2 Tail Call Optimization

Since **much** of a LISP involves having recursion, we would have a very cursed LISP indeed if we didn't handle tail calls. The main method of handling tail calls involves a special flag `tail` in the recursive evaluation function `eval`.

We use this to track whether or not a recursive descent is at the tail of a call chain. To see this clearly, consider the following (moral equivalent) implementation of `if`:

```
// where eval(sexp, image, tail) -> Value
Special::If => {
            let cond = sexp.nth(1);
            let then_branch = sexp.nth(2);
            let else_branch = sexp.nth(3);

            let c = eval(cond, image, false);
            if !is_falsy(c) {
                eval(then_branch, image, true)
            } else {
                eval(else_branch, image, true)
```

```
            }
}
```

Thus when we finally get to processing a closure call, we delay evaluation and return the value of the closure body whenever we encounter a closure call in the tail position (and indeed we *don't check that its the same closure*).

```
// eval(callee: Closure ..., tail) -> Value
if tail {
    Ok(Value::TailCall(c, arg_vals))
} else {
    call_closure(&c, arg_vals, image)
}
```

When we are not in a tail case (such as in the top-level execution, or whenever we return from this execution depth) then have a big ol' loop that rebinds which closure is currently being executed to save the stack space. Here's the moral equivalent of the closure call logic:

```
fn call_closure(callee: Closure, args, image) -> ... {
    let mut closure = callee;
    let mut args = args;

    loop {
        image.push_env(&closure.env); // restore captured
        image.push_frame(); // new space for params

        // bind params to args in the new frame
        closure
            .params
            .push_into(image)

        // body is always in tail position
        let result = eval(Rc::clone(&closure.body), image, true)?;

        image.pop_frame(); // unbind params
        image.pop_frame(); // unbind captured env

        match result {
            // the evaluation
            Value::TailCall(next_closure, next_args) => {
                closure = next_closure;
                args = next_args;
            }
            actual_value => return actual_value,
        }
    }
}
```

3.3 Memory Management

Given the above design of the LISP Image, we basically need to tell Rust how and where exactly its going to stick those BTrees of values. In our case, we use an off the shelf heap allocator with no dependencies instead of writing our own bump allocator to more efficient. And thus we can just tell Rust to stick its heap there with that allocator… but where exactly?

We tell the linker to link our binary against `pi-side/memory.ld`, which starts the heap whenever the program ends at a 32 byte aligned address. This grows upwards while the stack grows downwards at `0x08000000` (but, to be honest, we don't have much of a stack since its just the interpreter).

Note that this also doesn't give us much of a heap, since in `.rodata` we have all of Bad Apple bitwise dumped in the kernel binary, for… reasons.

4 Communication Protocol

The framing protocol of messages strings two frames on top of each other; first, our framing protocol takes a SEXP / evaluation output and frames it in the following manner:

```
[HEADER] * n (sync barrier, any number is possible)
0u32 0u32 0u32 0u32 (message header, 4 u32 zeros)
n: u32 (length of message in bytes)
[PAYLOAD]
0u32 0u32 0u32 0u32 (message footer, 4 u32 zeros)
[FOOTER] * n (sync barrier, any number is possible)
```

On the pi-side, it uses `0xdeadbeef` as the header and `0xfacefeed` as the footer; this is reversed in the unix-side. This design allows the last thing on the wire to be always the "discard any" part, and both sides would be subjectively "right" in terms of swapping roles in half-duplex.

After this inner frame, the message is then wrapped in a standard UART 8n1 frame which is then sent through the wire.

5 Hardware Features

This project is definitely three labs of content since it contains at least three labs worth of OS features reimplemented in LISP, and since we *are* programming a baremetal Pi after all we will talk about those features here.

5.1 Interpreter (Rust) Side

5.1.1 GPIO

First, to get UART to work, we need to be able to talk to the GPIO pins. Fortunately there isn't actually that many interesting / shocking things to get GPIO working in Rust once you have

```rust
#[inline(always)]
pub unsafe fn put32(addr: usize, value: u32) {
    unsafe { ::core::ptr::write_volatile(addr as *mut u32, value) }
}
```

```rust
#[inline(always)]
pub unsafe fn get32(addr: usize) -> u32 {
    unsafe { ::core::ptr::read_volatile(addr as *const u32) }
}
```

the rest is just window dressing and following the data sheet.

5.1.2 UART

Similar to GPIO its mostly just data sheet reading and sticking constants in `pi-side/src/comm/uart.rs`. Once it *is* implemented though in `shared/src/lib.rs` we expose a neat trait that allows you to have a very simplified API to send stuff along our framing protocol. In particular:

```rust
pub struct PiUart;

impl shared::Transport for PiUart {
    fn put8(&mut self, b: u8) { put8(b); }
    fn get8(&mut self) -> u8 { get8() }
    fn put32(&mut self, v: u32) { put32(v); }
    fn get32(&mut self) -> u32 { get32() }
    fn flush(&mut self) { flush_tx(); }
}
```

And once you have that, the `Framer` struct can take anything with a transport trait and talk using it. Thus the Pi-side (and unix side, for that matter) message passing API becomes dead simple:

```rust
let mut framer = Framer::pi_side(PiUart);
framer.send("PI_READY".as_bytes());
let _ = framer.recv();
```

5.1.3 Preemptive Threads

We allow threading, but have not done much with it other than verify that it works with the LISP image (we don't expose threading APIs to LISP side). In theory this would allow you to have separate threads and multiple interpreters.

We follow a very standard implementation with a stack-allocated Thread Control Block and a struct with every data register and `spsr`.

Not much, but hey, its an implementation of preemptive threads in Rust.

5.2 Driver (LISP) Side

5.2.1 GPIO (Again!)

To first prove that things mostly work, we reimplemented GPIO access as a LISP function. Its mostly data sheet reading still, but its nice to know that the semantics of our LISP does actually support this.

```
(defun gpio/set-function (pin func)
  (let (group (/ pin 10)
              offset (* (mod pin 10) 3))
    (let (sel (@get32 (nth group gpio/FSEL)))
      (begin
       (set sel (& (~ (<< 0b111 offset)) sel))
       (set sel (binor (<< func offset) sel))
       (@put32 (nth group gpio/FSEL) sel)))))
```

Notice the `@put32`, which is how we call syscalls from LISP. They don't actually have different semantics from special forms other than being implemented in a different file.

5.2.2 VideoCore Mailbox

This is where things become interesting. Beyond sexps, our LISP exposes a set of buffer operation special ops (see Indexed Operations below) that directly allocates and operates on regions of memory. This allows us to send delineated mailboxes with tags, as in:

```
(defun mbox/-write-tags (buf offset pairs)
  (if (nullp pairs)
      (putidx buf offset 0) ;; end tag
    (let (msg (car (car pairs))
              tx (nth 1 (car pairs)))
      (let (rx-size (nth mbox/RX-SIZE msg)
                    tx-size (nth mbox/TX-SIZE msg)
                    tag (nth mbox/TAG msg))
        (let (sized (max rx-size tx-size))
          (begin
           (putidx buf offset tag)
           (putidx buf (+ offset 1) sized)
           (putidx buf (+ offset 2) 0)
           (@zero32 buf (+ offset 3) (/ sized 4))
           (fillidx buf (+ offset 3) tx)
           (mbox/-write-tags buf (+ (+ offset 3) (/ sized 4))
                             (cdr pairs))))))))
```

Notice `putidx`, `fillidx`, etc., which are operators that can write to memory buffers such as `buf`. With this, we can implement the standard mailbox protocol to talk to the VideoCore and get serial numbers etc.

5.2.3 Framebuffer Video

Naturally if you have VideoCore working you want to put stuff on screen. At this point with the optimizations done above this is then mostly a matter of just writing really hard to the framebuffer. The only thing interesting here is the `@unpack1to16` syscall, which takes a 1 bit per pixel array (black and white) and expands it to a 16 bits per pixel array with the values `0x0000` and `0xFFFF`. This is useful for things like Bad Apple which of course is implemented and tested.

6 Language Features

6.1 Types

Type	Literal syntax	Description
Integer	`42`, `-7`	Signed 32-bit integer
Unsigned	`u42`	Unsigned 32-bit integer
Addr	`#0x20200000`	Raw memory address
Bool	`true`, `false`	Boolean
Nil	`nil`	Empty list / false
Cons	`'(1 2 3)`	Linked list (cons cells)
String	`"hello"`	String
Array	via `(fill n val)`	Mutable u32 vector (Rust `Vec<u32>`)
Closure	via `lambda` / `defun`	Function with captured environment
Macro	via `defmacro`	Syntax transformer

6.2 Number Arithmetic Rules

- `Int` op `Int` → `Int`
- `Unsigned` op `Unsigned` → `Unsigned`
- Mixed `Int`/`Unsigned` → `Unsigned` (promotes)
- `Addr` +/- `Int` or `Unsigned` → `Addr` (pointer offset)
- `Addr` - `Addr` → `Int` (distance)
- `Addr` * or / anything → error

6.3 Special Forms

6.3.1 Binding & Functions

Form	Description
`(set name value)`	Bind name to value. Mutates existing binding if any.
`(let (a 1 b 2) body)`	Local bindings; earlier bindings visible to later.
`(defun name (params) body)`	Define a named function. Desugars to set + lambda.
`(lambda (params) body)`	Create a closure. Alias: `fn`.
`(defmacro name (params) body)`	Define a macro.
`(macroexpand (macro-call))`	Expand a macro without executing.

6.3.2 Control Flow

Form	Description
`(if cond then else)`	Conditional. Evaluates then or else branch.
`(begin e1 e2 ... en)`	Evaluate forms in sequence, return last.

6.3.3 Arithmetic (binary, all return Number)

Form	Alias	Description
`(add a b)`	`+`	Addition
`(sub a b)`	`-`	Subtraction
`(mul a b)`	`*`	Multiplication
`(div a b)`	`/`	Division
`(mod a b)`	`%`	Modulo

6.3.4 Bitwise

Form	Alias	Description
`(lshift a b)`	`<<`	Left shift
`(rshift a b)`	`>>`	Right shift
`(binnot a)`	`~`	Bitwise NOT
`(binor a b)`	`|`	Bitwise OR
`(binand a b)`	`&`	Bitwise AND

6.3.5 Comparison

Form	Alias	Description
`(eq a b)`		Equality
`(gt a b)`	`>`	Greater than
`(lt a b)`	`<`	Less than
`(gte a b)`	`>=`	Greater than or equal
`(lte a b)`	`<=`	Less than or equal

6.3.6 Logic (short-circuit)

Form	Description
`(not a)`	Logical NOT.
`(and a b)`	If a is falsy return a, else evaluate and return b.
`(or a b)`	If a is truthy return a, else evaluate and return b.
`(xor a b)`	True iff exactly one is truthy.

6.3.7 List Operations

Form	Description
`(cons a b)`	Construct a cons cell.
`(car l)`	First element of a cons cell.
`(cdr l)`	Rest of a cons cell.
`(list a b ...)`	Variadic; construct a list from arguments.
`(nullp x)`	True if x is nil. Alias: `null?`.

6.3.8 Type Conversions

Form	Description
`(addr x)`	Convert number to Addr type.
`(signed x)`	Convert number to signed Integer.
`(unsigned x)`	Convert number to Unsigned.

6.3.9 Arrays

Form	Description
`(array list)`	Convert a list of numbers to an Array.
`(unpack array)`	Convert an Array back to a list.
`(full n value)`	Create a new Array of n copies of value.

6.3.10 Indexed Operations (work on both Array and Addr)

These operations accept either an Array or a heap Addr as the first argument. With an Array, they are bounds-checked and mutate the backing Vec in place. With an Addr (from `@alloc32` or similar), they read/write raw memory at `addr + index * 4`.

Form	Description
`(getidx target n)`	Read u32 at index n.
`(putidx target n val)`	Write u32 at index n.
`(readidx target offset n)`	Read n elements from offset into a list.
`(fillidx target offset list)`	Write list values starting at offset.
`(fullidx target offset n val)`	Fill n slots starting at offset with val.

6.4 Syscalls

Syscalls are prefixed with `@` and provide direct hardware access.

6.4.1 Memory-Mapped I/O (volatile, for device registers)

Syscall	Description
`(@get32 addr)`	Read u32 from address (volatile).
`(@put32 addr value)`	Write u32 to address (volatile).

6.4.2 Bulk Memory — list-based

These walk a lisp list and issue individual reads/writes. Parallel to the indexed ops above, but operate on raw addresses only.

Syscall	Description
`(@read32 addr offset n)`	Read n u32 slots → list. Starts at addr+offset*4.
`(@zero32 addr offset n)`	Zero n u32 slots starting at addr+offset*4.
`(@fill32 addr offset list)`	Write list values to consecutive u32 slots.
`(@full32 addr offset n val)`	Fill n u32 slots with val (`slice::fill`).

6.4.3 Bulk Memory — array-based (memcpy)

These use `copy_nonoverlapping` for maximum throughput. Parallel to the list-based ops above, but transfer entire Arrays in one shot.

Syscall	Description
`(@ldr src offset n)`	Load n u32 slots → Array. src: Addr or Array.
`(@str dst offset array)`	Store Array to dst+offset*4. dst: Addr or Array.

6.4.4 Memory Barriers

Syscall	Description
`(@dsb)`	Data synchronization barrier.
`(@prefetch_flush)`	Prefetch flush.

6.4.5 Heap Allocation

Syscall	Description
`(@alloc32 n [align])`	Allocate n u32 slots (zeroed). Returns addr.
`(@free32 addr n [align])`	Free a previous alloc32.

6.4.6 UART

Syscall	Description
`(@uart/init)`	Initialize UART.
`(@uart/put8 val)`	Write a byte to UART.
`(@uart/get8)`	Read a byte from UART (blocks).

6.4.7 Timing

Syscall	Description
`(@delay n)`	Busy-wait loop; n iterations of a nop.

6.4.8 Video

Syscall	Description
`(@unpack1to16 src dst)`	Expand 1bpp Array to 16bpp Array (`0x0000`/`0xFFFF`).

6.4.9 Parsing

Parsing a LISP is shockingly, not hard. We use `no_std` nom as the parser, and it mostly just works. Note that there are some special syntax beyond sexps which we support to identify things like syscalls vs normal functions / special forms etc. The exact list being:

```
(a b c)                     # list (cons chain ending in nil)
'(a b c)                    # quote sugar: desugars to (list a b c)
@name                       # syscall (case-insensitive): get32, put32, dsb,
prefetch_flush
#42  #0xFF  #0b101          # address literal (decimal, 0x hex, 0b binary)
u42  u0xFF  u0b101          # unsigned literal (decimal, 0x hex, 0b binary)
```

```
42  -7                      # integer (decimal)
0xFF                        # integer (hex)
0b1010                      # integer (binary)
"hello"                     # string (supports \n \t \\ \")
+ - * / % > < ~ | & << >>   # operator specials
nil true false              # literal values
defun lambda if ...         # named special forms
anything-else               # symbol
; comment                   # line comment (to end of line)
```